

← برگه سؤال را همراه دفترچه پاسخ تحویل دهید نام و نام خانوادگی:

پرسش ۱ (۲ نمره) – فرض کنید یک ساختمان داده جدید پیاده‌سازی کرده‌ایم. این که برای این ساختمان داده یک تکرارکننده (iterator) به سبک کتابخانه STL بنویسیم چه مزیتی خواهد داشت؟

این که تمام الگوریتم‌های موجود در کتابخانه اس‌تی‌ال می‌توانند روی ساختمان داده ما استفاده شوند.

پرسش ۲ (۲ نمره) – چرا در تعریف لیست‌های پیوندی گره‌های لیست را مخفی می‌کنیم (دسترسی به آن را به داخل کلاس لیست محدود می‌کنیم)؟ دو دلیل ذکر کنید.

۱. امکان تغییر در پیاده‌سازی بدون تغییر کدهای استفاده‌کننده

۲. عدم اجازه تغییر در لیست از طریق گره‌های لیست پیوندی

پرسش ۳ (۴ نمره) – (الف) فرض کنید کلاسی به نام A داریم که فقط یک سازنده بدون آرگومان دارد. این کلاس را طوری تغییر دهید که نتوان بیش از ۵ شیء از آن ساخت. در صورتی که تلاش برای ساختن بیش از این تعداد صورت پذیرد یک exception به نام TooManyAs رخ دهد! (راهنمایی: از اعضای static استفاده کنید).

(۲ نمره سازنده - ۰.۵ اعلان فیلد استاتیک - ۰.۵ تعریف فیلد استاتیک)

```
class A {
public:
    A() { if (count >= 5) throw TooManyAs(); count++; }
private:
    static int count;
};
int A::count = 0;
```

(ب) در پاسخ بخش (الف) تغییری ایجاد کنید که در کل اجرای برنامه بتوان بیش از ۵ شیء از A ساخت، اما در هر لحظه نتوان بیش از ۵ شیء داشت. (۱ نمره)

```
class A {
...
    ~A() { count--; }
...
};
```

پرسش ۴ (۲ نمره) – فرآیند ترجمه و اجرای برنامه زیر به کدامیک از دلایل زیر خاتمه می‌یابد؟ چرا؟

```
#include <iostream>
#define SIZE 1000

class A {
public:
    A() { data = new int[SIZE]; }
    A(A& a) {
        for (int i = 0; i < SIZE; i++)
            data[i] = a.data[i];
    }
    int get(int i) { return data[i]; }
private:
    int* data;
};

int main() {
    A x;
    A y = x;
    std::cout << x.get(10);
}
```

الف) به طور کامل ترجمه و اجرا می‌شود  
 ب) خطای ترجمه: پارامتر a باید ثابت (const) تعریف شود  
 پ) خطای ترجمه: عملگر = برای کلاس A تعریف نشده  
 ت) خطای ترجمه: دسترسی به فیلد خصوص a توسط شیء دیگر  
 ث) خطای زمان اجرا: دسترسی غیرمجاز به حافظه  
 در سازنده کپی برای data حافظه تخصیص نیافته  
 ج) خطای زمان اجرا: آزادکردن دوباره‌ی حافظه

---

پرسش ۵ (۴ نمره) – قطعه برنامه‌ی مقابل نمونه‌ای از استفاده از کلاس Point را نشان می‌دهد که برای کار با نقاط در فضای دوبعدی تعریف شده است. بخش عمومی (public) کلاس Point را بنویسید. (نیازی به نوشتن بدنه‌ی متدها نیست).

```
Point p(10, 2.3);
Point q;
p.translate(12.4, -3.1);
Point r = p.rotate(q, 30);
cout << p.x() << p.y() << endl;
```

```
class Point {
public:
    Point(double x = 0, double y = 0); // or a no-arg constructor
    void translate(double dx, double dy);
    Point rotate(Point p, double angle);
    double x();
    double y();
};
```

---

پرسش ۶ (۶ نمره) – تابع `count_even` تعداد عناصر زوج یک بردار اعداد صحیح را برمی‌گرداند:

```
int count_even(vector<int> v) {  
    int result = 0;  
    for (int i = 0; i < v.size(); i++)  
        if (v[i] % 2 == 0)  
            result++;  
    return result;  
}
```

الف) تابع فوق را تعمیم دهید (به نام `count_if`) طوری که یک بردار از `int` و یک اشاره‌گر به تابع از `int` به `bool` می‌گیرد و تعداد عناصری را برمی‌گرداند که تابع مذکور را `true` می‌کنند. به کمک این تابع تعداد عناصر زوج یک بردار را محاسبه کنید.

```
int count_if(vector<int> v, bool (*pred)(int)) {  
    int result = 0;  
    for (int i = 0; i < v.size(); i++)  
        if (pred(v[i]))  
            result++;  
    return result;  
}  
bool is_even(int x) { return x % 2 == 0 }  
count_even(v, is_even);
```

ب) به کمک الگوها تابع `count_if` بخش قبل را عمومی کنید طوری که تعریف آن محدود به `int` نباشد.

```
template<typename T>  
int count_if(vector<T> v, bool (*pred)(T)) {  
    int result = 0;  
    for (int i = 0; i < v.size(); i++)  
        if (pred(v[i]))  
            result++;  
    return result;  
}
```

پ) فرض کنید بخواهیم تابع قسمت قبل را علاوه بر `vector` روی تایپ‌های `set` یا `list` به کار ببریم. با توجه به این که این تایپ‌ها عملگر `[]` ندارند تعریف بخش قبل را طوری تغییر دهید که بتوان آن را به این منظور استفاده کرد.

```
template<typename I, typename P>  
int count_if(I first, I last, P pred) {  
    int result = 0;  
    while (first != last) {  
        if (P(*first))  
            result++;  
        first++;  
    }  
    return result;  
}
```